

PHƯƠNG PHÁP ĐỆ QUY (RECURSION)

PGS.TS. Đặng Thị Oanh

Ngày 23 tháng 2 năm 2026

Tài liệu tham khảo

- 1 Ian Parberry and W. Gasarch, **Problems on Agor** ebook, 2002.
- 2 Manuel Rubio-Sánchez, **Introduction to recursive programming**, CRC Press, 2018.

Review the Previous Lesson

- 1 Phân tích thuật toán
- 2 Độ phức tạp của thuật toán
- 3 Chứng minh tính đúng đắn của thuật toán

Nội dung

- 1 Khái niệm về đệ quy
- 2 Lựa chọn giải thuật đệ quy
- 3 Đánh giá độ phức tạp giải thuật đệ quy
- 4 Khử đệ quy
- 5 Bài tập

Khái niệm về đệ quy

- **Về bản chất:** đệ quy là cách định nghĩa một đối tượng dựa trên chính nó.

Khái niệm về đệ quy

- **Về bản chất:** đệ quy là cách định nghĩa một đối tượng dựa trên chính nó.
- **Trong Toán học:** phương pháp chứng minh quy nạp, chính là một nguyên lý đệ quy.

Khái niệm về đệ quy

- **Về bản chất:** đệ quy là cách định nghĩa một đối tượng dựa trên chính nó.
- **Trong Toán học:** phương pháp chứng minh quy nạp, chính là một nguyên lý đệ quy.
- **Trong lập trình:** một hàm đệ quy, chính là một hàm mà trong thân hàm có lời gọi đến chính nó.

Một số hình ảnh và ví dụ về đệ quy

- Hình ảnh tiêu biểu:



Một số hình ảnh và ví dụ về đệ quy

- Hình ảnh tiêu biểu:



Xem thêm tại đường Link:

<https://gifer.com/en/gifs/recursion>

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong tự nhiên: Tự nhiên sử dụng đệ quy để tối ưu hóa không gian và phát triển

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong tự nhiên: Tự nhiên sử dụng đệ quy để tối ưu hóa không gian và phát triển

- *Cây và lá:* Các cành nhỏ mọc ra từ cành lớn hơn, tuân theo quy luật tương tự, lá dương xỉ lớn chứa các lá nhỏ hơn có hình dạng giống hệt lá lớn.

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong tự nhiên: Tự nhiên sử dụng đệ quy để tối ưu hóa không gian và phát triển

- *Cây và lá:* Các cành nhỏ mọc ra từ cành lớn hơn, tuân theo quy luật tương tự, lá dương xỉ lớn chứa các lá nhỏ hơn có hình dạng giống hệt lá lớn.
- *Bông tuyết (Fractal):* Hình dạng bông tuyết được tạo thành từ các nhánh nhỏ liên tục lặp lại ở quy mô nhỏ hơn.

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong tự nhiên: Tự nhiên sử dụng đệ quy để tối ưu hóa không gian và phát triển

- *Cây và lá:* Các cành nhỏ mọc ra từ cành lớn hơn, tuân theo quy luật tương tự, lá dương xỉ lớn chứa các lá nhỏ hơn có hình dạng giống hệt lá lớn.
- *Bông tuyết (Fractal):* Hình dạng bông tuyết được tạo thành từ các nhánh nhỏ liên tục lặp lại ở quy mô nhỏ hơn.
- *Vỏ ốc và xoắn ốc.*

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong Toán học (Định nghĩa tự gọi):

Toán học sử dụng đệ quy để định nghĩa các dãy số hoặc hình ảnh phức tạp từ các trường hợp đơn giản nhất:

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong Toán học (Định nghĩa tự gọi):

Toán học sử dụng đệ quy để định nghĩa các dãy số hoặc hình ảnh phức tạp từ các trường hợp đơn giản nhất:

- *Dãy Fibonacci*: $F(n) = F(n - 1) + F(n - 2)$
với $F(0) = 1, F(1) = 1$.

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong Toán học (Định nghĩa tự gọi):

Toán học sử dụng đệ quy để định nghĩa các dãy số hoặc hình ảnh phức tạp từ các trường hợp đơn giản nhất:

- *Dãy Fibonacci*: $F(n) = F(n - 1) + F(n - 2)$ với $F(0) = 1$, $F(1) = 1$.
- *Giai thừa*:

$$n! = n * (n - 1)!$$

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong nghệ thuật (Vòng lặp hình ảnh):

Nghệ thuật sử dụng đệ quy để tạo ra cảm giác vô tận, chiều sâu hoặc sự lặp lại hoa văn:

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong nghệ thuật (Vòng lặp hình ảnh):

Nghệ thuật sử dụng đệ quy để tạo ra cảm giác vô tận, chiều sâu hoặc sự lặp lại hoa văn:

- *Hiệu ứng Droste:*

Một bức tranh xuất hiện bên trong chính nó, lặp đi lặp lại vô hạn lần.

Một số hình ảnh và ví dụ về đệ quy

Đệ quy trong nghệ thuật (Vòng lặp hình ảnh):

Nghệ thuật sử dụng đệ quy để tạo ra cảm giác vô tận, chiều sâu hoặc sự lặp lại hoa văn:

- *Hiệu ứng Droste:*

Một bức tranh xuất hiện bên trong chính nó, lặp đi lặp lại vô hạn lần.

- *Nghệ thuật Mandala:* Các hoa văn hình học phức tạp thường được tạo ra bằng cách lặp lại các hình cơ bản bên trong chính chúng.



Lược đồ giải thuật đệ quy

$\text{RecursiveAlgorithm}(P_n) \equiv // P_n - \text{parameter}(S)$

if $(P_n = P_0)$ // Trường hợp cơ sở

<statements;>

else // Trường hợp tổng quát

<statements;>

$\text{RecursiveAlgorithm}(P_{n-1})$

<statements;>

endif

End.

Lược đồ giải thuật đệ quy

$\text{RecursiveAlgorithm}(P_n) \equiv // P_n - \text{parameter}(S)$

if $(P_n = P_0)$ // Trường hợp cơ sở

$\langle \text{statements}; \rangle$

else // Trường hợp tổng quát

$\langle \text{statements}; \rangle$

$\text{RecursiveAlgorithm}(P_{n-1})$

$\langle \text{statements}; \rangle$

endif

End.

$$P_n \longleftarrow P_{n-1} \longleftarrow P_{n-2} \longleftarrow \cdots \longleftarrow P_0$$

Ví dụ - Calculate $n!$

S(n): int $\equiv$ // Recursive function return $n!$ value

if ($n == 0 || n == 1$)

$S = 1;$

else

$S = n * S(n-1);$

End.

Ví dụ - Calculate Fib(n)

Fib(n): int $\equiv$ // Recursive function return n^{th} fibonacci value

if ($n == 1 || n == 2$)
 Fib = 1;

else

 Fib = Fib(n-1) + Fib(n-2);

End.

Ví dụ - Tower of Hanoi

MoveTower(n, A, B, C) $\equiv$ // function move tower
n-disks from A to B

if ($n == 1$)

Move1Disk(A,B);

else

MoveTower($n-1, A, C, B$);

Move1Disk(A,B);

MoveTower($n-1, C, B, A$);

End.

Tính đúng đắn của giải thuật đệ quy

Chứng minh tính đúng đắn của giải thuật đệ quy dựa trên dựa trên **quy nạp toán học**

Tính đúng đắn của giải thuật đệ quy

Chứng minh tính đúng đắn của giải thuật đệ quy dựa trên dựa trên **quy nạp toán học**

Gồm hai bước chính:

chứng minh trường hợp cơ sở (base case) đúng và giả sử bài toán đúng với quy mô nhỏ hơn ($n - 1$) để chứng minh nó đúng với quy mô lớn hơn (n).

Tính đúng đắn của giải thuật đệ quy

Chứng minh tính đúng đắn của giải thuật đệ quy dựa trên dựa trên **quy nạp toán học**

Gồm hai bước chính:

chứng minh trường hợp cơ sở (base case) đúng và giả sử bài toán đúng với quy mô nhỏ hơn ($n - 1$) để chứng minh nó đúng với quy mô lớn hơn (n).

Phương pháp này đảm bảo lời giải bài toán lớn được suy ra từ các lời giải nhỏ.

Các bước chi tiết

- ➊ **Trường hợp cơ sở (Base Case):** Chứng minh giải thuật hoạt động đúng với trường hợp đơn giản nhất, thường là khi dừng đệ quy (ví dụ: $n = 0$ hoặc $(n = 1)$).

Các bước chi tiết

- ❶ **Trường hợp cơ sở (Base Case):** Chứng minh giải thuật hoạt động đúng với trường hợp đơn giản nhất, thường là khi dừng đệ quy (ví dụ: $n = 0$ hoặc $(n = 1)$).
- ❷ **Bước quy nạp (Recursive Step)**
 - **Giả thiết quy nạp:** Giả sử giải thuật hoạt động đúng cho các trường hợp nhỏ hơn (kích thước $< n$).
 - **Chứng minh:** Dựa vào giả thiết, chứng minh giải thuật cũng hoạt động đúng cho trường hợp hiện tại (n).

Các bước chi tiết

- ❶ **Trường hợp cơ sở (Base Case):** Chứng minh giải thuật hoạt động đúng với trường hợp đơn giản nhất, thường là khi dừng đệ quy (ví dụ: $n = 0$ hoặc $(n = 1)$).
- ❷ **Bước quy nạp (Recursive Step)**
 - **Giả thiết quy nạp:** Giả sử giải thuật hoạt động đúng cho các trường hợp nhỏ hơn (kích thước $< n$).
 - **Chứng minh:** Dựa vào giả thiết, chứng minh giải thuật cũng hoạt động đúng cho trường hợp hiện tại (n).
- ❸ **Kết luận:** Nếu cả 2 bước trên đúng, giải thuật đệ quy đúng với mọi trường hợp.

Ví dụ

Chứng minh hàm tính giai thừa $n!$

- **Cơ sở:** $0! = 1$ hoặc $1! = 1$ (đúng theo định nghĩa).
- **Quy nạp:** Giả sử tính đúng $(n - 1)!$. Ta có $n! = n * (n - 1)!$. Vì $(n - 1)!$ đúng, nên suy ra $n!$ đúng.

Hiểu rõ cấu trúc đệ quy và số lần gọi đệ quy là chìa khóa để xác định độ phức tạp trong trường hợp xấu nhất.

Độ phức tạp giải thuật đệ quy

- Đánh giá độ phức tạp giải thuật đệ quy thường dựa vào việc thiết lập và giải công thức truy hồi ($T(n)$) (thời gian chạy cho đầu vào cỡ n dựa trên các bài toán con nhỏ hơn).

Độ phức tạp giải thuật đệ quy

- Đánh giá độ phức tạp giải thuật đệ quy thường dựa vào việc thiết lập và giải **công thức truy hồi** ($T(n)$) (thời gian chạy cho đầu vào cỡ n dựa trên các bài toán con nhỏ hơn).
- **Các phương pháp chính:** substitution, cây đệ quy (recursion tree) và Master Theorem.

Độ phức tạp giải thuật đệ quy

- **Phương pháp Cây đệ quy:** Trực quan hóa quá trình đệ quy thành cây, tính tổng chi phí tại mỗi cấp và tổng chi phí toàn cây để tìm ra độ phức tạp tổng thể.

Độ phức tạp giải thuật đệ quy

- **Phương pháp Cây đệ quy:** Trực quan hóa quá trình đệ quy thành cây, tính tổng chi phí tại mỗi cấp và tổng chi phí toàn cây để tìm ra độ phức tạp tổng thể.
- **Công thức truy hồi:** Xác định chi phí của bước hiện tại và số lượng bài toán con.

Độ phức tạp giải thuật đệ quy

$$T(n) = \begin{cases} c & \text{if } n = n_0 \\ a.T(f(n)) + g(n) & \text{otherwise.} \end{cases}$$

- $T(n)$: Thời gian chạy của thuật toán với bài toán kích thước n ;
- c : Thời gian chạy với trường hợp cơ sở $T(n_0)$ (hằng số);
- a : Số lần gọi hàm đệ quy;
- $g(n)$: Thời gian xử lý khác.

Độ phức tạp giải thuật đệ quy

Một số ví dụ

- Thuật toán $S(n)$

$$T_S(n) = \begin{cases} 1 & \text{if } n = 1 \\ T_S(n-1) + 1 & \text{if } n > 1. \end{cases}$$

Độ phức tạp giải thuật đệ quy

Một số ví dụ

- Thuật toán $S(n)$

$$T_S(n) = \begin{cases} 1 & \text{if } n = 1 \\ T_S(n-1) + 1 & \text{if } n > 1. \end{cases}$$

- Thuật toán $Fib(n)$

$$T_{Fib}(n) = \begin{cases} 1 & \text{if } n = 1, 2 \\ T_{Fib}(n-1) + T_{Fib}(n-2) + 1 & \text{if } n > 2. \end{cases}$$

Độ phức tạp giải thuật đệ quy

Một số ví dụ

- Thuật toán $S(n)$

$$T_S(n) = \begin{cases} 1 & \text{if } n = 1 \\ T_S(n-1) + 1 & \text{if } n > 1. \end{cases}$$

- Thuật toán $Fib(n)$

$$T_{Fib}(n) = \begin{cases} 1 & \text{if } n = 1, 2 \\ T_{Fib}(n-1) + T_{Fib}(n-2) + 1 & \text{if } n > 2. \end{cases}$$

- Thuật toán $MoveTower(n, A, B, C)$

$$T_{Move}(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T_{Move}(n-1) + 1 & \text{if } n > 1. \end{cases}$$

Độ phức tạp giải thuật đệ quy

Use repeated substitution

$$T_S(n) = \begin{cases} 1 & \text{if } n = 1 \\ T_S(n-1) + 1 & \text{if } n > 1. \end{cases}$$

$$\begin{aligned} T_S(n) &= T_S(n-1) + 1 \\ &= T_S(n-2) + 1 + 1 \\ &= T_S(n-3) + 1 + 1 + 1 \\ &= \dots \\ &= T_S(n - (n-1)) + 1 + \dots + 1 \\ &= 1 + 1 \dots + 1 = n \\ &\rightarrow T_S(n) = O(n). \end{aligned}$$

Recursion Elimination - Khử đệ quy

Ưu của giải thuật đệ quy

Giải thuật đệ quy có ưu điểm nổi bật là giúp mã nguồn ngắn gọn, dễ đọc, dễ hiểu và logic, đặc biệt hiệu quả khi giải quyết các bài toán có cấu trúc phân cấp hoặc chia để trị như duyệt cây, đồ thị, Tháp Hà Nội hoặc sắp xếp. Đệ quy tuân theo tư duy tự nhiên, giảm độ phức tạp khi triển khai và hạn chế việc quản lý biến tạm thời.

Recursion Elimination - Khử đệ quy

Nhược điểm của giải thuật đệ quy

Nhược điểm chính của giải thuật đệ quy là tiêu tốn nhiều bộ nhớ (stack memory) do phải lưu trữ các trạng thái hàm gọi, dẫn đến nguy cơ tràn stack (stack overflow) khi số lần gọi quá lớn. Ngoài ra, đệ quy thường có hiệu suất thấp hơn so với vòng lặp vì tốn chi phí gọi hàm, tạo/xóa các biến cục bộ và khó gỡ lỗi (debug).

Recursion Elimination - Khử đệ quy

Nhược điểm của giải thuật đệ quy

Nhược điểm chính của giải thuật đệ quy là tiêu tốn nhiều bộ nhớ (stack memory) do phải lưu trữ các trạng thái hàm gọi, dẫn đến nguy cơ tràn stack (stack overflow) khi số lần gọi quá lớn. Ngoài ra, đệ quy thường có hiệu suất thấp hơn so với vòng lặp vì tốn chi phí gọi hàm, tạo/xóa các biến cục bộ và khó gỡ lỗi (debug).

⇒ Hạn chế sử dụng giải thuật đệ quy đối với các bài toán có quy mô dữ liệu lớn.

Recursion Elimination - Khử đệ quy

Nhược điểm của giải thuật đệ quy

Nhược điểm chính của giải thuật đệ quy là tiêu tốn nhiều bộ nhớ (stack memory) do phải lưu trữ các trạng thái hàm gọi, dẫn đến nguy cơ tràn stack (stack overflow) khi số lần gọi quá lớn. Ngoài ra, đệ quy thường có hiệu suất thấp hơn so với vòng lặp vì tốn chi phí gọi hàm, tạo/xóa các biến cục bộ và khó gỡ lỗi (debug).

⇒ Hạn chế sử dụng giải thuật đệ quy đối với các bài toán có quy mô dữ liệu lớn.

⇒ KHỬ ĐỆ QUY

Recursion Elimination - Khử đệ quy

Khử đệ quy

- Là quá trình chuyển đổi thuật toán đệ quy (hàm tự gọi lại chính nó) thành thuật toán không đệ quy (thường dùng vòng lặp) nhằm tối ưu hóa hiệu suất, giảm thiểu tiêu thụ bộ nhớ (ngăn xếp - stack) và tránh tràn bộ nhớ. Kỹ thuật này thường được áp dụng bằng cách sử dụng vòng lặp while/for hoặc tự định nghĩa stack (Stack dữ liệu) để lưu trữ trạng thái thay vì dựa vào stack hệ thống.

Recursion Elimination - Khử đệ quy

Khử đệ quy

- Là quá trình chuyển đổi thuật toán **đệ quy** (hàm tự gọi lại chính nó) thành thuật toán **không đệ quy** (thường dùng vòng lặp) nhằm **tối ưu hóa hiệu suất**, giảm thiểu tiêu thụ bộ nhớ (ngăn xếp - stack) và tránh tràn bộ nhớ. Kỹ thuật này thường được áp dụng bằng cách sử dụng **vòng lặp while/for** hoặc **tự định nghĩa stack** (Stack dữ liệu) để lưu trữ trạng thái thay vì dựa vào stack hệ thống.
- **Các dạng phổ biến:** Đệ quy đuôi (tail recursion), đệ quy tuyến tính (linear recursion) và đệ quy đa nhánh (Multiple Recursion - Fibonacci).

Khử đệ quy một số dạng thường gặp

Đệ quy đuôi (tail recursion)

Nếu lời gọi đệ quy là hành động cuối cùng, có thể khử bằng cách thay thế bởi vòng lặp. Đây là trường hợp đơn giản nhất.

Khử đệ quy một số dạng thường gặp

Đệ quy đuôi (tail recursion)

Nếu lời gọi đệ quy là hành động cuối cùng, có thể khử bằng cách thay thế bởi vòng lặp. Đây là trường hợp đơn giản nhất.

Cách khử: Dùng vòng lặp và một biến lưu trữ KQ tạm thời

Đệ quy - $n!$

```
S(n): int ≡  
    if (n == 0 || n == 1)  
        S = 1;  
    else  
        S = n * S(n-1);  
  
End.
```

Khử đệ quy - $n!$

```
S(n): int ≡  
    GT = 1;  
    for i = 2 to n  
        GT = GT * i;  
    return GT  
End.
```


Multiple Recursion

Hàm gọi lại chính nó nhiều lần trong 1 lần thực thi.

Cách khử: Sử dụng kỹ thuật Quy hoạch động (Dynamic Programming) - lưu kết quả các bước trước vào mảng hoặc biến để tránh tính lại.

Đệ quy - Fibonacci

Fib(n): int $\equiv$

if ($n == 1 || n == 2$)

Fib = 1;

else

Fib = Fib(n-1)
+ Fib(n-2);

End.

Khử ĐQ - Fibonacci

Fib(n): int $\equiv$

a = 1;

b = 0;

Temp = [];

for i = 1 to n

Temp = a;

a = a + b;

b = Temp;

Endfor

return b

End.

Recursion Elimination - Khử đệ quy

Chú ý

Không phải mọi thuật toán đệ quy đều dễ dàng chuyển đổi sang vòng lặp. Đệ quy vẫn được ưa chuộng khi cần mã nguồn gọn gàng, dễ hiểu, đặc biệt trong các bài toán phức tạp như sắp xếp nhanh (Quick Sort) hay duyệt cây.

Bài tập

- Đánh giá độ phức tạp của thuật toán đệ quy;
- Lập trình đệ quy;
- Đặt bài toán, thiết kế, phân tích và triển khai thuật toán.